

LANGCHAIN

I. Introduction

LangChain is an open-source framework designed to simplify the creation of applications using large language models (LLMs). It provides a standard interface for chains, lots of integrations with other tools, and end-to-end chains for common applications. It allows AI developers to develop applications based on the combined Large Language Models (LLMs) such as GPT-4 with external sources of computation and data. This framework comes with a package for both Python and JavaScript.



Figure 1. LangChain

LangChain follows a general pipeline where a user asks a question to the language model where the vector representation of the question is used to do a similarity search in the vector database and the relevant information is fetched from the vector database and the response is later fed to the language model. further, the language model generates an answer or takes an action.

1. Applications of LangChain

LangChain is a powerful tool that can be used to build a wide range of LLM-powered applications. It is simple to use and has a large user and contributor community.

- Document analysis and summarization
- **Chatbots:** LangChain can be used to build chatbots that interact with users naturally. For example, LangChain can be used to build a chatbot that can answer client questions, provide customer assistance, and even arrange appointments.
- **Code analysis:** LangChain can be used to analyse code and find potential bugs or security flaws.
- **Answering questions using sources:** LangChain can be used to answer questions using a variety of sources, including text, code, and data. For example, LangChain can be used to answer questions about a specific topic by searching through a variety of sources, such as Wikipedia, news articles, and code repositories.
- **Data augmentation:** LangChain can be used to augment data by generating new data that is similar to existing data. For example, LangChain can be used to generate new text data that is similar to existing text data. This can be useful for training machine learning models or for creating new datasets.

- **Text classification:** LangChain can be used for text classifications and sentiment analysis with the text input data
- **Text summarization:** LangChain can be used to summarize the text in the specified number of words or sentences.
- **Machine translation:** LangChain can be used to translate the input text data into different languages.

2. LangChain Key Concepts

The main properties of LangChain Framework are:

- **Components:** Components are modular building blocks that are ready and easy to use to build powerful applications. Components include LLM Wrappers, Prompt Template and Indexes for relevant information retrieval.
- **Chains:** Chains allow us to combine multiple components together to solve a specific task. Chains make it easy for the implementation of complex applications by making it more modular and simple to debug and maintain.
- **Agents:** Agents allow LLMs to interact with their environment. For example, using an external API to perform a specific action.

II. Setting up the environment

Note: These docs focus on the JavaScript LangChain library.

LangChain simplifies every stage of the LLM application lifecycle:

- **Development:** Build your applications using LangChain's open-source building blocks, components, and third-party integrations. Use LangGraph.js to build stateful agents with first-class streaming and human-in-the-loop support.
- **Productionization:** Use LangSmith to inspect, monitor and evaluate your chains, so that you can continuously optimize and deploy with confidence.
- **Deployment:** Till now LangGraph Cloud currently only support for Python so we gonna skip this step

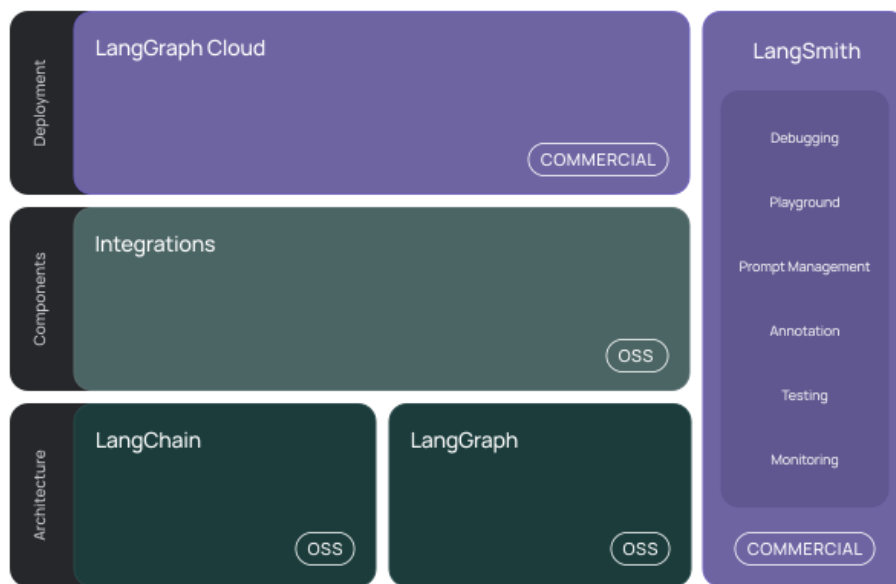


Figure 2. LangChain Step

Concretely, the framework consists of the following open-source libraries:

`@langchain/core`: Base abstractions and LangChain Expression Language.

`@langchain/community`: Third party integrations.

Partner packages (e.g. `@langchain/openai`, `@langchain/anthropic`, etc.): Some integrations have been further split into their own lightweight packages that only depend on `@langchain/core`.

`langchain`: Chains, agents, and retrieval strategies that make up an application's cognitive architecture.

`LangGraph.js`: Build robust and stateful multi-actor applications with LLMs by modeling steps as edges and nodes in a graph.

`LangSmith`: A developer platform that lets you debug, test, evaluate, and monitor LLM applications.

1. Setup

Let's get started with build a simple ChatBot

1.1. Install LangChain (using NodeJS)

```
npm i langchain
```

LangChain is written in TypeScript and can be used in:

- Node.js (ESM and CommonJS) - 18.x, 19.x, 20.x
- Cloudflare Workers
- Vercel / Next.js (Browser, Serverless and Edge functions)
- Supabase Edge Functions
- Browser
- Deno
- Bun

However, note that individual integrations may not be supported in all environments.

While this package acts as a sane starting point to using LangChain, much of the value of LangChain comes when integrating it with various model providers, datastores, etc. By default, the dependencies needed to do that are NOT installed. You will need to install the dependencies for specific integrations separately. We'll show how to do that in the next sections of this guide.

Please also see the section on [installing integration packages](#) for some special considerations when installing LangChain packages.

```
npm package.json
{
  "name": "your-project",
  "version": "0.0.0",
  "private": true,
  "engines": {
    "node": ">=18"
  },
  "dependencies": {
    "@langchain/anthropic": "^0.0.2",
    "langchain": "0.0.207"
  },
  "overrides": {
    "@langchain/core": "0.1.5"
  }
}
```

1.2. LangSmith

Many of the applications you build with LangChain will contain multiple steps with multiple invocations of LLM calls. As these applications get more and more complex, it becomes crucial to be able to inspect what exactly is going on inside your chain or agent. The best way to do this is with LangSmith.

After you sign up at the link above, make sure to set your environment variables to start logging traces:

```
export LANGCHAIN_TRACING_V2="true"
export LANGCHAIN_API_KEY="..."

# Reduce tracing latency if you are not in a serverless environment
# export LANGCHAIN_CALLBACKS_BACKGROUND=true
```

2. Quickstart

First up, let's learn how to use a language model by itself. LangChain supports many different language models that you can use interchangeably – in this documentation we gonna use OpenAI!

Install dependencies:

```
npm i @langchain/openai
```

Add environment variables:

```
OPENAI_API_KEY=your-api-key
```

Instantiate the model:

```
import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({
  model: "gpt-3.5-turbo", //you can change to the model that necessary or fit w ur project
  temperature: 0.8 //this one may be change following the model
});
```

Let's first use the model directly. ChatModels are instances of LangChain “Runnables”, which means they expose a standard interface for interacting with them. To just simply call the model, we can pass in a list of messages to the `.invoke` method.

```
import { HumanMessage } from "@langchain/core/messages";

await model.invoke([new HumanMessage({ content: "Hi! I'm Thuan" })]);
```

```

AIMessage {
  "id": "chatcmpl-A64of8iD4GIFNSYlOaFHxPdCeyl9E",
  "content": "Hi Bob! How can I assist you today?",
  "additional_kwargs": {},
  "response_metadata": {
    "tokenUsage": {
      "completionTokens": 10,
      "promptTokens": 11,
      "totalTokens": 21
    },
    "finish_reason": "stop"
  },
  "tool_calls": [],
  "invalid_tool_calls": [],
  "usage_metadata": {
    "input_tokens": 11,
    "output_tokens": 10,
    "total_tokens": 21
  }
}

```

The model on its own does not have any concept of state. For example, if you ask a followup question:

```
await model.invoke([new HumanMessage({ content: "What's my name?" })]);
```

```

AIMessage {
  "id": "chatcmpl-A64ogC7owxmPla3ggZERNCFZpVHSp",
  "content": "I'm sorry, but I don't have access to personal information about users unless it has been shared with me in the course of our conversation. If you'd like to tell me your name, feel free!",
  "additional_kwargs": {},
  "response_metadata": {
    "tokenUsage": {
      "completionTokens": 39,
      "promptTokens": 11,
      "totalTokens": 50
    },
    "finish_reason": "stop"
  },
  "tool_calls": [],
  "invalid_tool_calls": [],
  "usage_metadata": {
    "input_tokens": 11,
    "output_tokens": 39,
    "total_tokens": 50
  }
}

```

Let's take a look at the example [LangSmith trace](#)

We can see that it doesn't take the previous conversation turn into context, and cannot answer the question. This makes for a terrible chatbot experience!

To get around this, we need to pass the entire conversation history into the model. Let's see what happens when we do that:

```
import { AIMessage } from "@langchain/core/messages";

await model.invoke([
  new HumanMessage({ content: "Hi! I'm Bob" }),
  new AIMessage({ content: "Hello Bob! How can I assist you today?" }),
  new HumanMessage({ content: "What's my name?" }),
]);
```

```
AIMessage {
  "id": "chatcmpl-A64ohhg3P4Buliw8mUCLi3zYHNOvS",
  "content": "Your name is Bob! How can I help you today, Bob?",
  "additional_kwargs": {},
  "response_metadata": {
    "tokenUsage": {
      "completionTokens": 14,
      "promptTokens": 33,
      "totalTokens": 47
    },
    "finish_reason": "stop"
  },
  "tool_calls": [],
  "invalid_tool_calls": [],
  "usage_metadata": {
    "input_tokens": 33,
    "output_tokens": 14,
    "total_tokens": 47
  }
}
```

And now we can see that we get a good response!

This is the basic idea underpinning a chatbot's ability to interact conversationally. So how do we best implement this?

3. Message History

We can use a Message History class to wrap our model and make it stateful. This will keep track of inputs and outputs of the model, and store them in some datastore. Future interactions will then load those messages and pass them into the chain as part of the input. Let's see how to use this!

We import the relevant classes and set up our chain which wraps the model and adds in this message history. A key part here is the function we pass into as the `getSessionHistory()`. This function is expected to take in a `sessionId` and return a Message History object. This `sessionId` is used to distinguish between separate conversations, and should be passed in as part of the config when calling the new chain.

Let's also create a simple chain by adding a prompt to help with formatting:

```
// We use an ephemeral, in-memory chat history for this demo.
import { InMemoryChatMessageHistory } from "@langchain/core/chat_history";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { RunnableWithMessageHistory } from "@langchain/core/runnables";

const messageHistories: Record<string, InMemoryChatMessageHistory> = {};

const prompt = ChatPromptTemplate.fromMessages([
  [
    "system",
    `You are a helpful assistant who remembers all details the user shares with you.`,
  ],
  ["placeholder", "{chat_history}"],
  ["human", "{input}"],
]);

const chain = prompt.pipe(model);

const withMessageHistory = new RunnableWithMessageHistory({
  runnable: chain,
  getMessageHistory: async (sessionId) => {
    if (messageHistories[sessionId] === undefined) {
      messageHistories[sessionId] = new InMemoryChatMessageHistory();
    }
    return messageHistories[sessionId];
  },
  inputMessagesKey: "input",
  historyMessagesKey: "chat_history",
});
```


We now need to create a config that we pass into the runnable every time. This config contains information that is not part of the input directly, but is still useful. In this case, we want to include a `session_id`. This should look like:

```
const config = {
  configurable: {
    sessionId: "abc2",
  },
};

const response = await withMessageHistory.invoke(
  {
    input: "Hi! I'm Bob",
  },
  config
);

response.content;
```

```
"Hi Bob! How can I assist you today?"
```

```
const followupResponse = await withMessageHistory.invoke(
  {
    input: "What's my name?",
  },
  config
);

followupResponse.content;
```

```
"Your name is Bob. How can I help you today?"
```

Great! Our chatbot now remembers things about us. If we change the config to reference a different `session_id`, we can see that it starts the conversation fresh.

```
const config2 = {
  configurable: {
    sessionId: "abc3",
  },
};

const response2 = await withMessageHistory.invoke({
  input: "What's my name?",
}, config2);

response2.content;
```

"I'm sorry, but I don't have your name. If you tell me, I'll remember it for our future conversations"... 1 more character

However, we can always go back to the original conversation (since we are persisting it in a database)

```
const config3 = {
  configurable: {
    sessionId: "abc2",
  },
};

const response3 = await withMessageHistory.invoke(
  {
    input: "What's my name?",
  },
  config3
);

response3.content;
```

"Your name is Bob. What would you like to talk about?"

This is how we can support a chatbot having conversations with many users!

4. Streaming (More research is needed before applying!)

Now we've got a functional chatbot. However, one *really* important UX consideration for chatbot application is streaming. LLMs can sometimes take a while to respond, and so in order to improve the user experience one thing that most application do is stream back each token as it is generated. This allows the user to see progress.

It's actually super easy to do this!

All chains expose a `.stream()` method, and ones that use message history are no different. We can simply use that method to get back a streaming response.

The results below are what is being desired to be successful:

```
| Hi  
| Todd  
| !  
| Here  
| 's  
| a  
| joke  
| for  
| you  
| :  
|
```

```
| Why  
| did  
| the  
| scare  
| crow  
| win  
| an  
| award  
| ?  
|
```

```
| Because  
| he  
| was  
| outstanding  
| in  
| his  
| field  
| !  
|
```

5. Conversational RAG

5.1. Overview

In many Q&A applications we want to allow the user to have a back-and-forth conversation, meaning the application needs some sort of “memory” of past questions and answers, and some logic for incorporating those into its current thinking.

In this guide we focus on **adding logic for incorporating historical messages**.

We will cover two approaches:

1. Chains, in which we always execute a retrieval step;
2. Agents, in which we give an LLM discretion over whether and how to execute a retrieval step (or multiple steps).

For the external knowledge source, we will use the same [LLM Powered Autonomous Agents](#) blog post by Lilian Weng from the [RAG tutorial](#).

5.2. Setup

5.2.1. [Dependencies](#)

5.2.2. [Initial setup](#)

5.2.3. [Contextualizing the question](#)

5.2.4. [Chain with Chat History](#)

1 References

- [1]. [What are large language models \(LLMs\)?](#)
- [2]. Myer, Mike. [“Are Generative AI and Large Language Models the Same Thing?”](#) Quiq, 12 May 2023
- [3]. Sheng, Ellen. [“In generative AI legal Wild West, the courtroom battles are just getting started”](#) CNBC, April 3, 2023
- [4]. [Getty Images Statement](#), Jan 17 2023 (Accessed June 29, 2023)